ConsentMesh

Système de Sensibilisation à la Confidentialité

Une salle qui impose sa propre politique de confidentialité — en matériel

Cahier des Charges Technique

Projet ESP32 — Système embarqué distribué
avec détection d'anomalies par apprentissage automatique

Date limite :	1er juin 2025
Version :	ConsentMesh v4
Statut :	Document de référence

Houssem Khouja Miled Trabelssi Salim Hamdi Yazid Rebai

ICE — Février 2026

Document confidentiel — Usage interne uniquement

Table des matières

1 Présentation du Projet **3**
1.1 Contexte et Problématique . . . 3
1.2 Proposition de Valeur . . . 3
1.3 Ce que le Système N'est Pas . . . 3

2 Architecture Générale du Système **3**
2.1 Vue d'Ensemble des Couches . . . 4
2.2 Topologie du Maillage . . . 4

3 Description des Couches Logicielles **4**
3.1 Couche 1 — Extraction de Caractéristiques (Nœuds ESP32) . . . 4
3.1.1 Vecteur d'Activité Local . . . 4
3.1.2 Contournement de la Randomisation MAC . . . 5
3.2 Couche 2 — Consensus Byzantine-Résilient . . . 5
3.2.1 Motivation . . . 5
3.2.2 Protocole . . . 5
3.2.3 Tolérance aux Pannes . . . 6
3.3 Couche 3 — Détection d'Anomalies par ML . . . 6
3.3.1 Approche . . . 6
3.3.2 Entraînement . . . 6
3.3.3 Inférence en Temps Réel . . . 6
3.4 Couche 4 — Sorties Publiques . . . 7
3.4.1 Affichage HDMI . . . 7
3.4.2 Bandeau LED WS2812B . . . 7

4 Spécifications Matérielles **7**
4.1 Architecture des Nœuds . . . 7
4.1.1 4 × Nœuds de Détection (ESP32-WROOM-32) . . . 7
4.1.2 1 × Nœud Pont (ESP32-WROOM-32) . . . 8
4.1.3 1 × Nœud d'Agrégation et d'Inférence (Raspberry Pi) . . . 8
4.2 Liste Complète du Matériel . . . 8

5 Spécifications Logicielles **9**
5.1 Environnement de Développement . . . 9
5.2 Protocoles de Communication . . . 10
5.3 Pipeline de Données . . . 10

6 Gestion des Frontières de la Salle **10**

7 Alimentation et Déploiement **11**
7.1 Démarrage du Système . . . 11
7.2 Configuration de Démonstration . . . 11
7.3 Démonstration de la Tolérance aux Pannes . . . 11

8 Limites Honnêtes du Système **12**

9 Répartition des Tâches **12**

10 Planning Prévisionnel **13**

11 Question Fondamentale **13**

1 Présentation du Projet

1.1 Contexte et Problématique

Chaque réunion confidentielle, chaque séance de thérapie, chaque discussion en salle de conseil partage la même vulnérabilité invisible : n'importe quel participant pourrait enregistrer. Il n'existe actuellement aucun mécanisme technique pour faire respecter une politique d'interdiction d'enregistrement. La confiance est le seul mécanisme disponible.

L'activité sans fil remplit constamment chaque salle. Les appareils émettent des balises, des rafales, des poignées de mains et des flux de données. Ces informations existent déjà dans l'air ambiant. Jusqu'à présent, elles sont invisibles pour les personnes qu'elles concernent le plus.

ConsentMesh rend cette activité visible.

1.2 Proposition de Valeur

Phrase clé du projet

Un maillage auto-surveillant de nœuds ESP32 observe l'environnement sans fil d'une salle, établit un consensus sur ce qu'il détecte, apprend ce qui est normal, et rend l'invisible visible — publiquement, en continu, et automatiquement.

1.3 Ce que le Système N'est Pas

Limites déclarées

- **Pas un brouilleur** — aucun signal n'est bloqué ou interféré
- **Pas un système de surveillance** — aucun individu n'est identifié
- **Pas un détecteur d'enregistrement certifié** — le système détecte des comportements sans fil anormaux, pas la preuve d'un enregistrement
- **Pas un système de stockage** — aucune donnée n'est conservée au-delà de la session en cours
- **Pas connecté au cloud** — tout le traitement s'effectue localement dans la salle

2 Architecture Générale du Système

Le système est structuré en quatre couches logicielles superposées sur deux niveaux matériels distincts.

2.1 Vue d'Ensemble des Couches

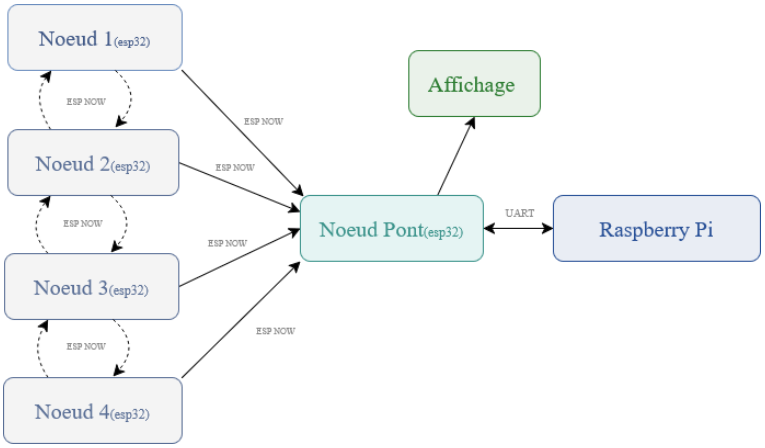
COUCHE 4 — Sortie publique
Affichage HDMI + Bandeau LED WS2812B

COUCHE 3 — Détection d'anomalies ML
Modèle IsolationForest sur nœud Raspberry Pi

COUCHE 2 — Consensus Byzantine-Résilient
Agrégation médiane + détection de nœuds défaillants

COUCHE 1 — Observation RF Passive
4 nœuds ESP32 en mode moniteur 802.11 passif

2.2 Topologie du Maillage



3 Description des Couches Logicielles

3.1 Couche 1 — Extraction de Caractéristiques (Nœuds ESP32)

Chaque nœud de détection opère en **mode moniteur 802.11 passif** et en **scan Bluetooth passif**. Il capture les métadonnées des trames sans fil sans lire leur contenu ni associer l'activité à des individus.

3.1.1 Vecteur d'Activité Local

Toutes les quelques secondes, chaque nœud calcule un **Vecteur d'Activité** contenant :

Caractéristique	Description
Compteur de trames	Nombre de trames 802.11 capturées par intervalle
Volume de trafic	Octets estimés transmis dans la fenêtre d'observation
Fréquence de rafales	Nombre et durée des pics d'émission
Durée de transmission	Durée continue des émissions par source
Score de continuité	Correspondance numéro de séquence entre adresses MAC successives
Empreinte de sonde	Hash des éléments d'information (IE) des requêtes de sonde
Signature temporelle	Intervalles caractéristiques entre transmissions

3.1.2 Contournement de la Randomisation MAC

Les appareils modernes (iOS 14+, Android 10+, Windows 10+) randomisent leurs adresses MAC. ConsentMesh utilise trois techniques complémentaires pour estimer le nombre réel d'appareils physiques distincts :

— **Continuité du numéro de séquence** — certains appareils ne réinitialisent pas ce compteur lors d'un changement de MAC

— **Empreinte de requête de sonde** — la combinaison des éléments IE est souvent spécifique à un chipset

— **Signature temporelle** — les intervalles d'émission sont partiellement déterminés par le pilote matériel

La métrique reportée est appelée *sources de transmission actives* et non *nombre d'appareils* pour refléter honnêtement cette incertitude.

3.2 Couche 2 — Consensus Byzantine-Résilient

3.2.1 Motivation

Dans un système de détection distribué à enjeux élevés en matière de confidentialité, un nœud défaillant ou compromis alimentant des données erronées constitue une défaillance de confiance. Si l'affichage ne peut pas être cru, toute la promesse du système s'effondre.

3.2.2 Protocole

Le protocole d'agrégation Byzantine-résilient léger s'inspire des principes BFDN (*Byzantine Fault Detection Network*) :

1. Chaque nœud diffuse son Vecteur d'Activité à tous les autres nœuds via ESP-NOW
2. Le nœud d'agrégation calcule la **médiane** sur toutes les lectures pour chaque dimension
3. Un **score de déviation** est calculé pour chaque nœud par rapport au consensus courant

4. Les nœuds dont la déviation dépasse un seuil sont marqués comme **potentiellement défaillants** et leur poids est réduit
5. Les nœuds anormaux de façon persistante sont mis en **quarantaine** jusqu'au retour à un comportement attendu

3.2.3 Tolérance aux Pannes

Avec $N = 4$ nœuds de détection et $f = 1$ nœud défaillant toléré :

$$N \geq 3f + 14 \geq 3(1) + 1 = 4$$

Limite déclarée

Une défaillance simultanée de deux nœuds ou plus brise la condition $N \geq 3f + 1$ et compromet la fiabilité du consensus. Cette limite est affichée en temps réel sur l'écran public via l'indicateur de santé du maillage.

3.3 Couche 3 — Détection d'Anomalies par ML

3.3.1 Approche

Plutôt que de tenter d'identifier un enregistrement — ce qu'aucune observation RF passive ne peut faire de manière fiable — le modèle apprend ce qui est *normal* dans cette salle et signale les déviations statistiques.

3.3.2 Entraînement

- Le modèle est entraîné **hors ligne** sur des données de référence collectées dans la salle lors de périodes d'inactivité connues
- Architecture : **IsolationForest** de scikit-learn ou petit réseau à propagation directe quantifié en INT8
- Contrainte mémoire : modèle $\leq$ 20 Mo pour tenir sur le Raspberry Pi Zero 2W
- Le modèle entraîné est déployé sur le Pi via flash de la carte SD

3.3.3 Inférence en Temps Réel

- Le nœud d'agrégation reçoit le vecteur de consensus et calcule un **Score d'Anomalie** continu
- Ce score est normalisé entre 0 (normal) et 1 (très anormal)
- Il pilote directement la couleur du bandeau LED et les indicateurs de l'affichage public
- La référence courante est mise à jour de façon incrémentale pendant les périodes d'inactivité entre les sessions

3.4 Couche 4 — Sorties Publiques

3.4.1 Affichage HDMI

Indicateur	Description
Sources actives	Nombre de sources de transmission sans fil détectées par consensus
Niveau d'activité	Inactif / Actif / Élevé, piloté par le Score d'Anomalie ML
Indicateurs comportementaux	Signaux anonymisés lors de transmission continue anormale
Santé du maillage	Nœuds actifs / Nœuds signalés défaillants
Durée de session	Temps écoulé depuis le début de l'activité détectée

3.4.2 Bandeau LED WS2812B

Couleur	État	Signification
Vert	Inactif	Appareils présents, activité de transmission minimale
Ambre	Actif	Comportement normal d'utilisation des appareils
Rouge	Élevé	Un ou plusieurs appareils présentent un comportement anormal

Réinitialisation : L'affichage et le bandeau se réinitialisent automatiquement lorsque la salle est détectée vide. Aucun historique n'est conservé.

4 Spécifications Matérielles

4.1 Architecture des Nœuds

4.1.1 4 × Nœuds de Détection (ESP32-WROOM-32)

- Mode de fonctionnement : moniteur 802.11 passif + scan Bluetooth passif
- Capteur : PIR HC-SR501 (détection d'occupation)
- Actuateur : écran OLED SSD1306 0,96" I2C (état de santé du nœud)
- Communication : ESP-NOW pair-à-pair vers tous les autres nœuds
- Alimentation : banque USB 10 000 mAh (autonomie $\geq$ 8 heures)
- Placement : coins de la salle, montés en hauteur

4.1.2 1 × Nœud Pont (ESP32-WROOM-32)

- Rôle : traduction de protocole ESP-NOW → UART série
- Aucun capteur attaché
- Placé physiquement à proximité du Raspberry Pi
- Alimenté via le port USB du Raspberry Pi

4.1.3 1 × Nœud d'Agrégation et d'Inférence (Raspberry Pi)

- Reçoit les Vecteurs d'Activité via liaison série depuis le nœud pont
- Exécute l'algorithme de consensus Byzantine-résilient
- Exécute l'inférence du modèle ML en temps réel
- Pilote l'affichage public via HDMI
- Contrôle le bandeau LED via GPIO (pin de données avec résistance 470Ω)
- Alimentation : adaptateur secteur USB 5V 2A
- Démarrage autonome : script Python lancé au boot via `systemd`

4.2 Liste Complète du Matériel

Composant	Qté	Rôle	Coût (USD)
Calcul			
ESP32-WROOM-32 Dev Board	5	4 nœuds détection + 1 pont	5–8 / u
Raspberry Pi Zero 2W	1	Agrégation, ML, affichage	~15
MicroSD 16 Go+	1	OS Raspberry Pi + code	~5
Capteurs			
PIR HC-SR501	4	Détection d'occupation	1–2 / u
Capteur magnétique de porte	1	Détection d'entrée	2–4
Actionneurs et Affichage			
Bandeau LED WS2812B 1m (60 LED)	1	Indicateur d'activité principal	8–12
OLED SSD1306 0,96" I2C	4	Santé par nœud	2–3 / u
Moniteur HDMI	1	Affichage public	prêté
Adaptateur Mini HDMI vers HDMI	1	Pi Zero vers moniteur	3–5
Alimentation			
Banque USB 10 000 mAh+	4	Nœuds coins sans fil	10–15 / u
Adaptateur secteur USB 5V 2A	1	Nœud d'agrégation	~5

Suite sur la page suivante...

Composant	Qté	Rôle	Coût (USD)
Alimentation 5V 2A (bandeau LED)	1	Bandeau LED	5–8
Câbles Micro-USB / USB-C	5	Alimentation générale	2–3 / u
Câblage et Interconnexion			
Pack de fils de liaison	1	Connexions GPIO	~5
Breadboards demi-taille	4	Prototypage capteurs	2–3 / u
Résistance 470Ω	1	Protection données LED	<1
Condensateur 1000 µF	1	Lissage alimentation LED	<1
Fil de connexion 22AWG	1 rouleau	Câblage capteur de porte	3–5
Boîtiers (optionnels)			
Boîtiers de projet / imprimés 3D	4	Protection nœuds coins	3–8 / u
Bandes velcro / adhésif de montage	1 pack	Fixation aux murs	3–5
Colliers de câbles	1 pack	Gestion des câbles	~2

Budget estimé total

Calcul	50–65 USD
Capteurs	10–15 USD
Actionneurs	20–30 USD
Alimentation	60–85 USD
Câblage	15–20 USD
Boîtiers	15–35 USD
Total	**~170–250 USD**

5 Spécifications Logicielles

5.1 Environnement de Développement

Cible	Environnement
ESP32 (nœuds + pont)	Arduino IDE ou ESP-IDF, langage C/C++
Raspberry Pi	Python 3.10+, scikit-learn, NumPy, pyserial, RPi.GPIO
Affichage	Pygame ou Tkinter pour le rendu de l'interface publique
ML (entraînement)	Jupyter Notebook sur ordinateur portable, export pickle

5.2 Protocoles de Communication

Liaison	Protocole
Nœud ↔ Nœud (maillage)	ESP-NOW pair-à-pair, canal dédié
Nœud pont → Raspberry Pi	UART série via USB, 115200 baud
Raspberry Pi → Affichage	HDMI
Raspberry Pi → Bandeau LED	GPIO PWM (WS2812B nécessite 800kHz)

5.3 Pipeline de Données

1. Les nœuds ESP32 capturent les trames en mode moniteur passif
2. Chaque nœud calcule son Vecteur d'Activité local toutes les 3 secondes
3. Les vecteurs sont diffusés entre pairs via ESP-NOW
4. Le nœud pont reçoit tous les vecteurs et les transmet au Pi en série
5. Le Pi exécute l'agrégation Byzantine-résiliente (calcul de médiane + scoring de déviation)
6. Le vecteur de consensus résultant entre dans le modèle ML
7. Le Score d'Anomalie pilote les sorties LED et affichage
8. L'affichage et le bandeau se réinitialisent à zéro occupation détectée

6 Gestion des Frontières de la Salle

Le RF ne respecte pas les murs physiques. Un nœud ESP32 placé dans une salle captera des signaux provenant du couloir adjacent ou de la pièce voisine. ConsentMesh adresse cette limitation de la façon suivante :

— **Scoring de proximité par consensus** — un appareil genuinement dans la salle sera visible par la majorité des nœuds. Un appareil extérieur ne sera visible que par le(s) nœud(s) le plus proche d'une paroi. Le nombre de nœuds détectant une empreinte donnée sert de score de confiance de présence.

— **Phase de calibration initiale** — lors de l'installation, un appareil connu parcourt le périmètre de la salle. Le système enregistre l'enveloppe RSSI à chaque nœud et établit les seuils de présence.

— **Absorption par la référence ML** — les signaux extérieurs constants (ex. : téléphone dans un bureau adjacent) sont absorbés dans la référence de base lors de la phase d'entraînement et ne déclenchent pas d'anomalie. Seul un appareil inconnu apparaissant à la frontière élèverait le score.

La métrique de présence reste étiquetée *sources actives estimées* pour refléter honnêtement cette incertitude résiduelle.

7 Alimentation et Déploiement

7.1 Démarrage du Système

L'ensemble du système démarre de façon autonome. Aucun PC, aucun terminal, aucune intervention humaine n'est nécessaire après la mise sous tension initiale :

— **Nœuds ESP32** : le firmware est flashé une seule fois. À la mise sous tension, le programme s'exécute automatiquement depuis la mémoire flash interne.

— **Raspberry Pi** : le script Python principal est enregistré en tant que service `systemd` et démarre au boot de l'OS.

— **Temps de démarrage estimé** : $\leq$ 15 secondes depuis la mise sous tension jusqu'à l'affichage actif.

7.2 Configuration de Démonstration

Élément	Description
4 nœuds de coin	Boîtiers sans fil avec banque USB, fixés aux coins de la salle
1 nœud pont	Placé près du Pi, alimenté par le port USB du Pi
1 Raspberry Pi	Près de l'écran, alimentation secteur, HDMI vers moniteur
1 bandeau LED	Monté de façon visible, alimenté séparément
1 capteur de porte	Câblé au nœud de coin le plus proche

7.3 Démonstration de la Tolérance aux Pannes

Pour démontrer le consensus Byzantine-résilient en direct : débrancher physiquement l'un des nœuds de coin pendant la démo. L'affichage s'adapte sans interruption et le nœud manquant apparaît comme signalé dans l'indicateur de santé du maillage. Ce moment tend à être mémorable pour un jury technique.

8 Limites Honnêtes du Système

Limitation	Impact et Atténuation
Pas de détection certifiée d'enregistrement	Limitation fondamentale du RF passif. Le système détecte des anomalies comportementales, pas la preuve d'enregistrement. Présenté comme une caractéristique de transparence.
Frontières de salle approximatives	Signaux extérieurs constants absorbés dans la référence ML. Scoring de proximité par consensus réduit les faux positifs.
Randomisation MAC	Nombre d'appareils estimé, pas certifié. Continuité des numéros de séquence et empreinte de sonde améliorent l'estimation. Métrique étiquetée honnêtement.
Tolérance $f = 1$ seulement	Deux pannes simultanées brisent le consensus. Affiché en temps réel via l'indicateur de santé.
RAM limitée sur Pi Zero 2W	Contraint la complexité du modèle ML. IsolationForest de scikit-learn reste dans les limites.

9 Répartition des Tâches

Membre	Responsabilités
#1	Firmware ESP32 nœuds de détection — mode moniteur, extraction de Vecteur d'Activité, randomisation MAC
#2	Firmware ESP32 maillage ESP-NOW — diffusion pair-à-pair, nœud pont, protocole série vers Pi
#3	Raspberry Pi — algorithme de consensus Byzantine-résilient, pipeline ML, inférence en temps réel
#4	Affichage public, contrôle bandeau LED, intégration système, calibration de salle, démonstration

10 Planning Prévisionnel

Semaine	Jalons
S1	Acquisition du matériel, configuration des environnements de développement, premier flash ESP32
S2	Mode moniteur passif fonctionnel sur un nœud, extraction du Vecteur d'Activité de base
S3	Maillage ESP-NOW multi-nœuds fonctionnel, nœud pont transmettant au Pi
S4	Algorithme de consensus Byzantine-résilient opérationnel, collecte de données de référence pour ML
S5	Entraînement et déploiement du modèle ML, affichage public et bandeau LED fonctionnels
S6	Intégration complète, calibration de salle, tests de tolérance aux pannes, préparation de la démo
1 juin	**Démonstration finale**

11 Question Fondamentale

La question que la salle pose

Ce système n'est pas de la surveillance. C'est de la contre-surveillance.
La surveillance observe les personnes au profit d'une institution. ConsentMesh observe la salle au profit de tous ses occupants. Il ne rapporte à personne. Il ne conserve rien. Il tient simplement un miroir face à l'environnement sans fil qui nous entoure déjà et dit : *voici ce qui se passe dans cette salle en ce moment, et voici dans quelle mesure nous sommes certains de vous dire la vérité.*
Cet acte — rendre l'invisible visible, et rendre cette visibilité elle-même digne de confiance — est une déclaration politique et technique.

À qui appartient l'air d'une salle ?

ConsentMesh v4 — Maillage ESP32 — Capteurs PIR et porte — Consensus Byzantine-Résilient — Détection d'anomalies ML embarquée — Éphémère, Transparent, Auto-surveillant